

STOCKOUT FIREWALL

Technical Architecture & System Design Documentation

AI Procurement & Fulfillment Resilience Network for India's Small Merchants

“When one supplier fails, the merchant shouldn't.”

Architecture • AI Orchestration • Optimization • Failover • Fulfillment • Payments • Security • MVP • Evaluation

Submission-ready technical documentation

Document Scope

This document presents the complete technical architecture and system design for Stockout Firewall, preserving the concept, terminology, architecture, features, technical direction, and source framing while organizing it into a judge-ready engineering document.

Part	Coverage
System Definition	Overview, problem, objective, core principle
Architecture	Layered architecture and service boundaries
AI & Language	Multilingual understanding, SKU normalization, orchestration
Procurement Intelligence	Supplier intelligence, reliability, landed cost, allocation
Resilience	Supplier failover, state machine, dynamic replanning
Fulfillment	Routing, consolidation, confidence, counterfactuals
FinTech	Cash-aware procurement, payment orchestration
Engineering	Database, APIs, events, security, auditability
MVP & Demo	Prototype scope, data model, demo flow, evaluation
Scale & Vision	Predictive stockout, collective procurement, network intelligence

1 System Overview

Stockout Firewall is a multilingual AI procurement and fulfillment orchestration platform designed for India's small merchants.

The system allows a merchant to communicate a complete wholesale requirement through WhatsApp, text, or voice. The platform converts the merchant's natural-language request into a structured procurement basket, normalizes products/SKUs, discovers suitable suppliers, evaluates supplier reliability, calculates total landed cost, optimizes supplier allocation, generates a feasible delivery plan, and provides a single merchant-side confirmation and payment experience.

The critical differentiator is dynamic resilience. The system does not assume that a supplier confirmation remains valid.

- Inventory may decrease.
- A supplier may reject an order.
- A supplier may provide only partial quantity.
- A supplier may change price.
- A supplier may become delayed.
- A delivery route may become infeasible.

When such an event occurs, the system detects the event, calculates the resulting shortfall, searches the supplier network for alternatives, reallocates the remaining quantity, and re-optimizes fulfillment.

Core system principle: Procurement is treated as a dynamic reliability and optimization problem rather than a static ordering transaction.

Product flow:



2 Problem Definition

2.1 Existing Merchant Workflow



The underlying issue is not simply lack of an ordering application. It is **supply-chain fragility**.

- Stockouts and partial availability
- Unexpected price changes
- Delivery failures and delays
- Fragmented orders and multiple supplier relationships
- Multiple deliveries and payments
- Supplier dependency
- Delayed discovery of shortages

3 System Objective

Stockout Firewall transforms manual procurement coordination into AI-assisted dynamic procurement orchestration.

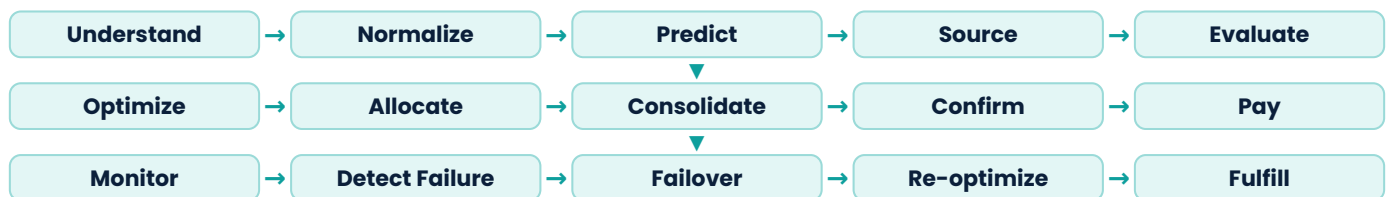
- Understand natural merchant language.
- Convert speech/text into structured intent.
- Normalize products and SKU representations.
- Discover suppliers.
- Evaluate supplier reliability.
- Determine actual available inventory.
- Optimize supplier allocation.
- Calculate total landed cost.
- Consider merchant liquidity and payment terms.
- Optimize fulfillment.
- Monitor supplier/order events.
- Detect failures.
- Automatically initiate failover.
- Recalculate allocation.
- Re-optimize delivery.
- Maintain merchant policy constraints.
- Provide explainable recommendations.
- Never hallucinate inventory or fulfillment.
- Maintain an auditable order lifecycle.

4 Core System Principle

Traditional Procurement

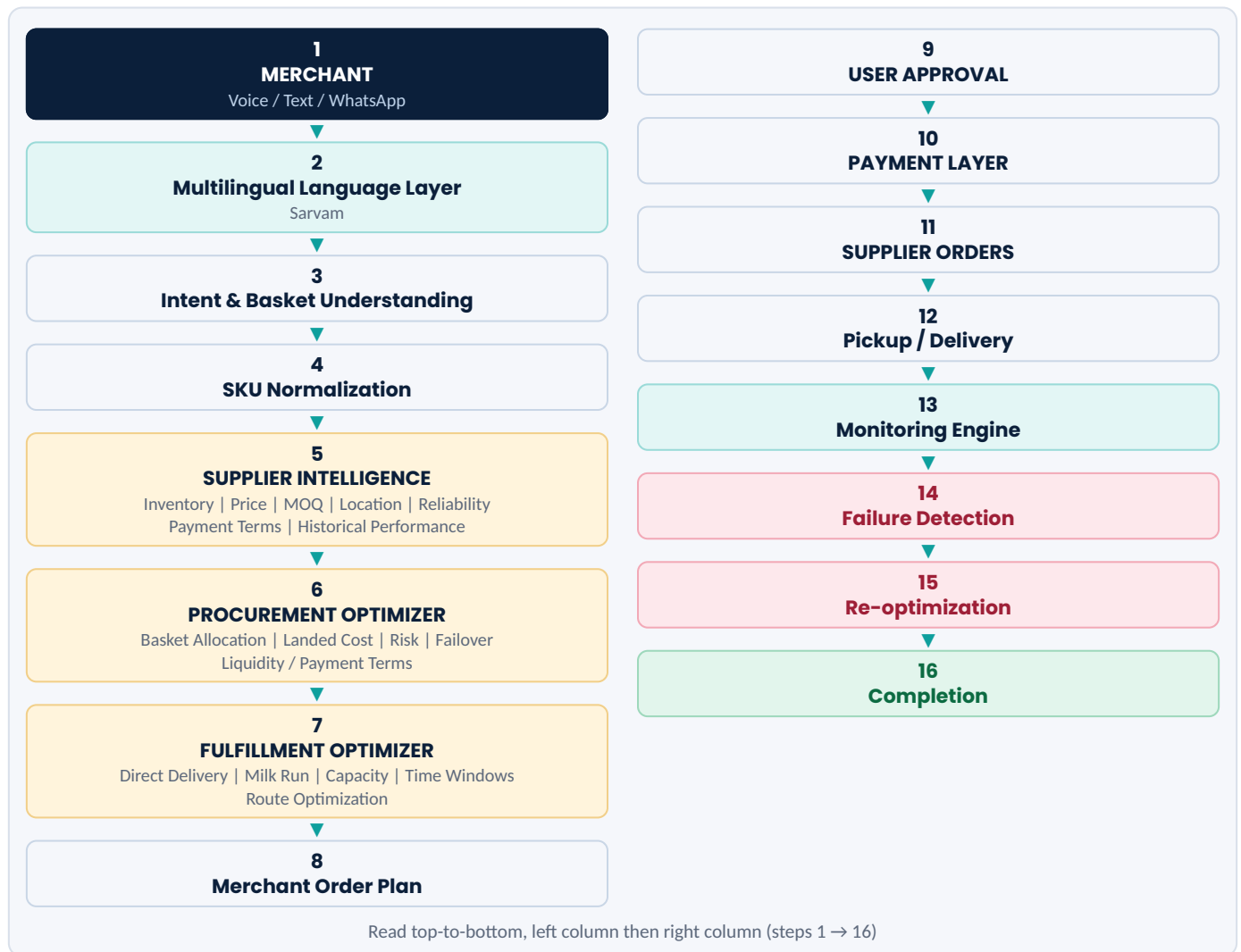


Stockout Firewall



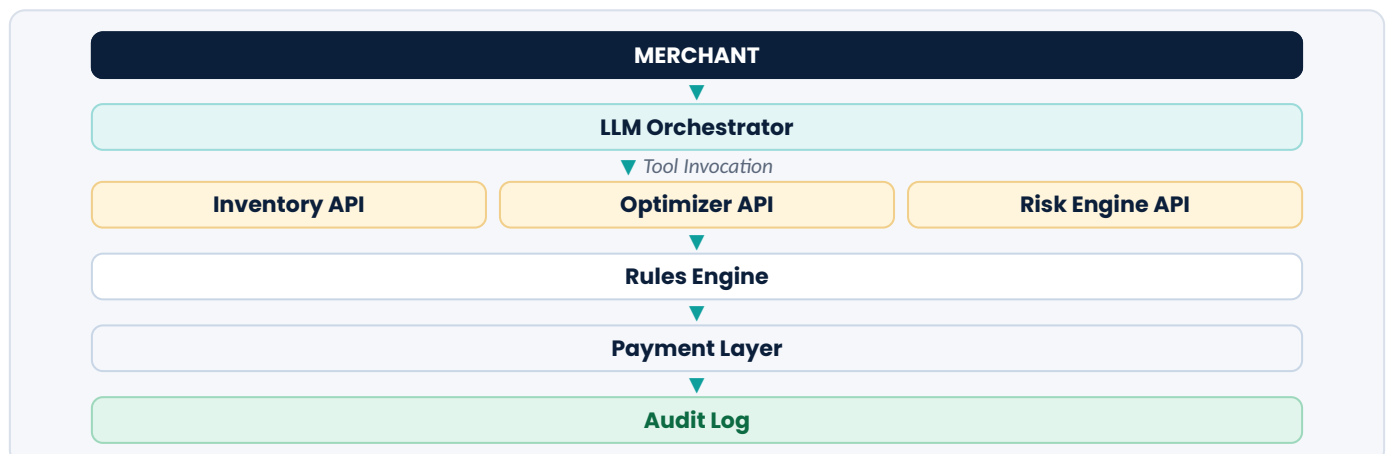
The platform does not treat procurement as completed when an order is placed. The procurement plan remains dynamically manageable until fulfillment is complete.

5 High-Level System Architecture



6 Architecture Principle: LLM ≠ Decision Engine

The LLM is responsible for understanding and orchestrating. Deterministic systems are responsible for facts and execution.

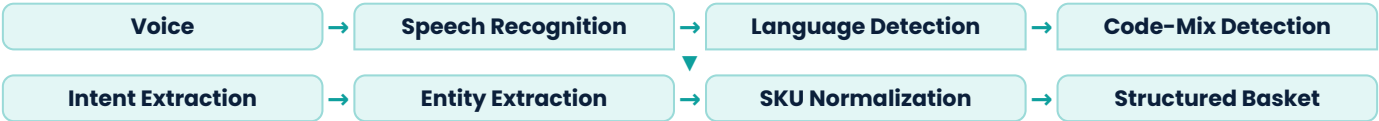


The LLM must **never** invent product price, inventory, supplier availability, payment status, delivery status, or supplier reliability.

7 Module 1 — Multilingual Order Understanding

The objective is to let merchants communicate naturally without learning a complex procurement interface.

Pipeline



Example input

"Anna, 5 dabba Fortune oil, 10 packet biscuit, 50 kilo rice beku."

```
{
  "items": [
    {"product": "Fortune Oil", "quantity": 5, "unit": "unit"},
    {"product": "Biscuits", "quantity": 10, "unit": "packet"},
    {"product": "Rice", "quantity": 50, "unit": "kg"}
  ]
}
```

8 Module 2 – SKU Normalization

Merchant language is not standardized. Different expressions may refer to the same underlying product.

Merchant expressions

"Fortune 1 litre 20 pieces"
"Fortune oil 20 bottles"
"20 bottles Fortune 1L"

Canonical representation

Brand = Fortune
Category = Cooking Oil
Pack Size = 1L
Quantity = 20
Total Volume = 20L

Canonical Product Schema

```
{
  "product_id": "PROD-001",
  "brand": "Fortune",
  "category": "Cooking Oil",
  "pack_size": 1,
  "unit": "L",
  "quantity": 20,
  "total_volume": 20
}
```

9 Module 3 – Supplier Intelligence

Every supplier receives a dynamic operational profile.

Metric	Supplier A Example
Rice Price	₹48/kg
Available Stock	60 kg
Distance	2.3 km
On-Time Fulfillment	97%
Cancellation	1.4%
Shortage Rate	1.8%
Response Time	4 min
MOQ	20 kg
Payment Terms	Immediate

Cheapest supplier does not necessarily mean best supplier. The optimizer evaluates **effective procurement quality** rather than raw price.

10 Supplier Reliability Model

- Availability Accuracy
- On-Time Fulfillment
- Cancellation Rate
- Shortage Rate
- Quality Dispute Rate
- Response Speed
- Price Stability



11 Module 4 – Total Landed Cost Engine

The platform must not optimize only product price.

Effective Cost =
 Product Cost
 + Delivery Cost
 + Consolidation Cost
 + Delay Risk
 + Failure Risk
 + Payment/Cash Cost
 + Supplier Complexity

Example**Supplier A**

Product	₹4,800
Delivery	₹300
Risk penalty	₹100
Effective Cost	₹5,200

Supplier B

Product	₹5,000
Delivery	₹100
Risk penalty	₹20
Effective Cost	₹5,120

Supplier B can therefore be the better choice despite a higher product price.

12 Module 5 — Basket Optimization

For a basket such as 100 kg Rice, 50 L Oil and 20 Biscuit Cartons, the optimizer searches for an allocation satisfying requirements while minimizing the selected objective.

Required

Rice	100 kg
Oil	50 L
Biscuit Cartons	20

Potential allocation

A	60 kg Rice + 20 L Oil
B	40 kg Rice
C	30 L Oil + 20 Biscuit Cartons
D	0

The optimizer considers price, inventory, reliability, MOQ, location, deadline, delivery capacity, payment terms and merchant preferences.

13 Optimization Formulation

A conceptual objective function is:

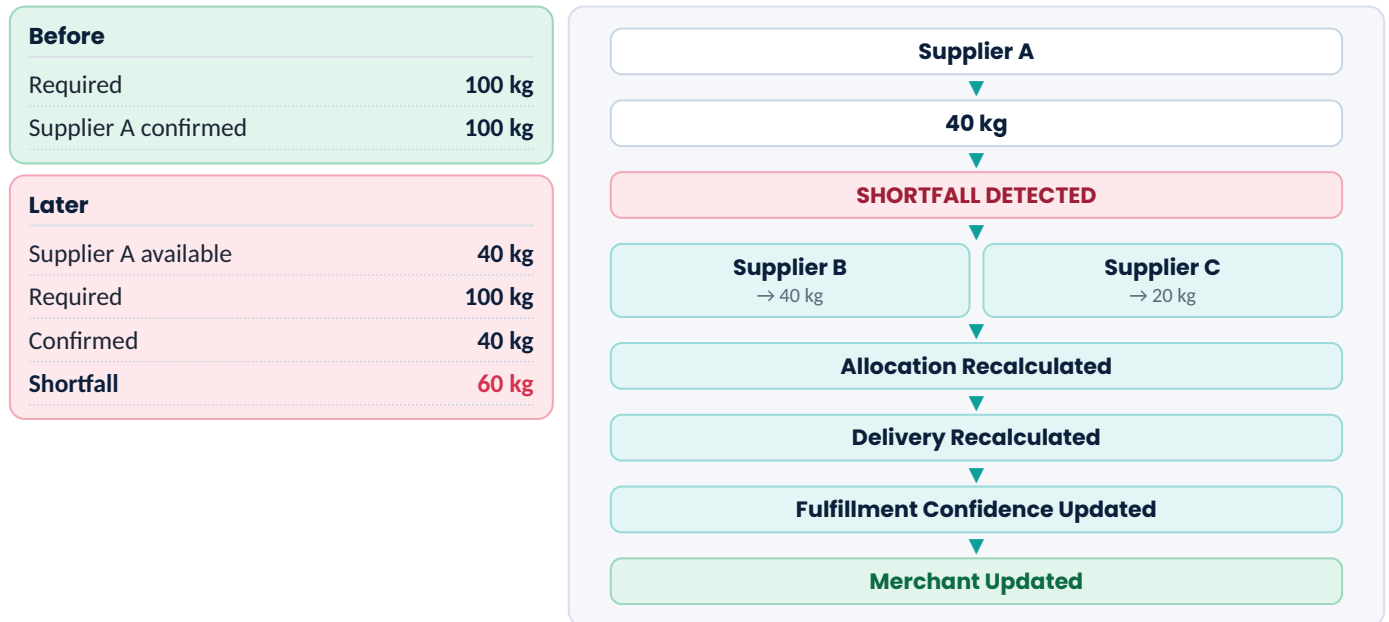
$$\min Z = C_{\text{product}} + C_{\text{delivery}} + P_{\text{delay}} + P_{\text{failure}} + P_{\text{supplier-count}} + C_{\text{liquidity}}$$

Subject to:

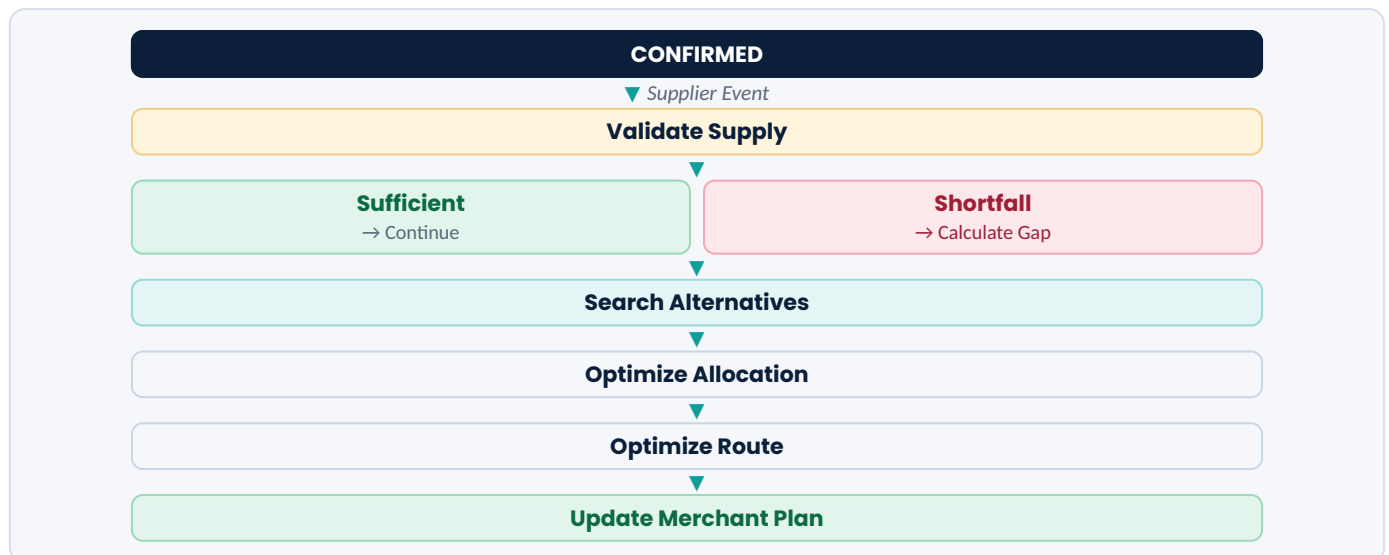
- AllocatedQuantity(p) ≥ RequiredQuantity(p)
- Allocated Quantity ≤ Supplier Inventory
- MOQ constraints satisfied
- Merchant Budget ≤ Maximum Budget
- Vehicle Load ≤ Vehicle Capacity
- Delivery ≤ Merchant Deadline
- Supplier Operating Hours respected
- Substitution rules respected
- Merchant procurement policies respected

This establishes the central engine as a genuine constraint optimization problem rather than an LLM-generated recommendation.

14 Module 6 — Supplier Failover Engine

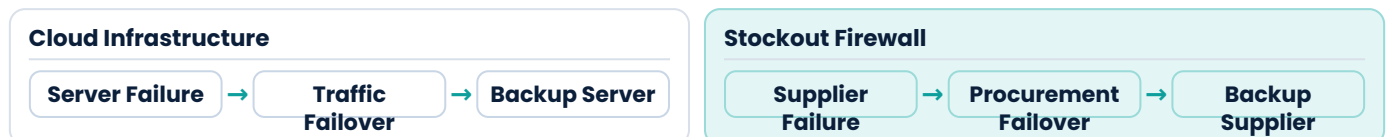


15 Failover State Machine



16 Why This Is Failover Infrastructure

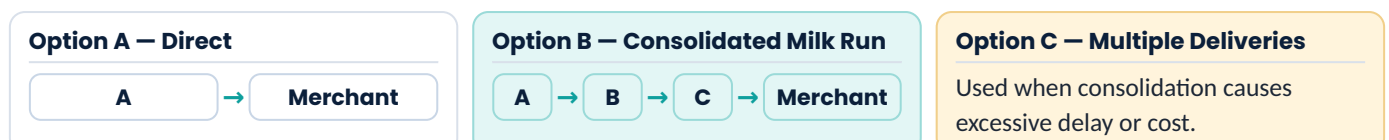
The architectural analogy is cloud infrastructure:



The system can therefore be positioned as failover infrastructure for fragmented wholesale procurement.

17 Module 7 — Delivery Optimization

Supplier allocation alone is insufficient. If the system chooses several suppliers, it must also determine how those orders should physically reach the merchant.



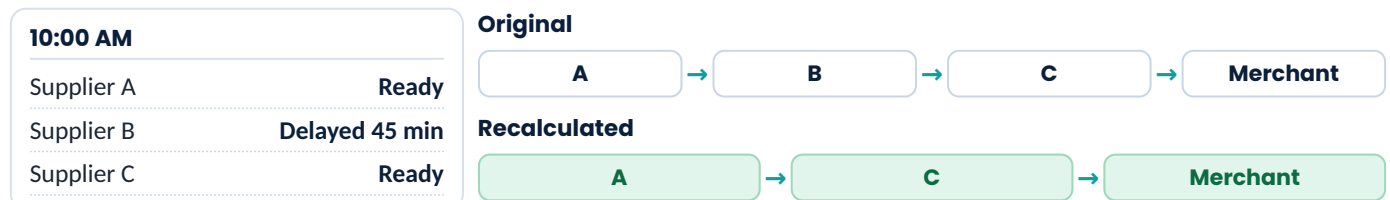
One delivery is an optimization outcome, not a hard promise.

18 Route Optimization

- Supplier Location
- Merchant Deadline
- Pickup Time
- Delivery Cost
- Merchant Location
- Vehicle Capacity
- Delivery Windows
- Traffic / Time
- Package Weight
- Number of Stops
- Supplier Readiness
- Package Volume
- Distance

For the prototype, Google OR-Tools / CP-SAT can be used for Vehicle Routing Problems, capacity constraints and time windows.

19 Dynamic Replanning



The route is not immutable. It can be recalculated when supplier readiness, deadlines or feasibility changes.

20 Fulfillment Confidence Engine

Before confirmation, the system displays a fulfillment confidence value derived from inventory confidence, supplier reliability, delivery feasibility, historical performance and deadline feasibility.

● 96% — High Confidence

● 78% — Medium Confidence

20 kg rice depends on secondary supplier confirmation.

The objective is not merely to generate a number, but to expose **why** confidence changes.

21 Explainable Procurement

Supplier B selected because:

- ✓ 7% better fulfillment reliability
- ✓ 1.8 km closer
- ✓ Inventory confirmed
- ✓ 7-day payment terms
- ✓ Landed cost only ₹80 higher than the cheapest option

The system should explain supplier selection in merchant-understandable terms.

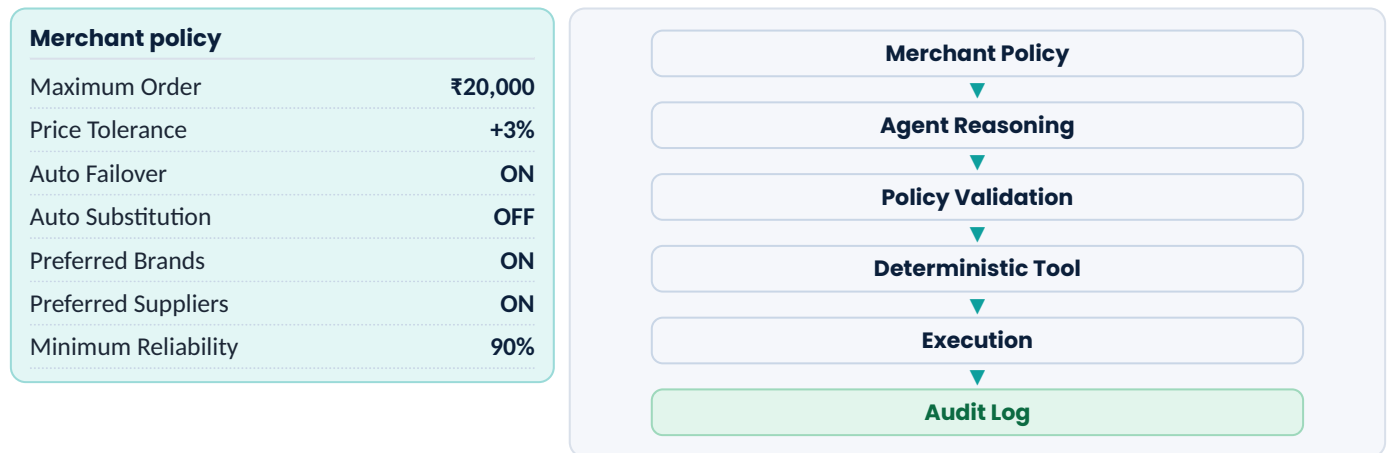
22 Counterfactual Procurement

Strategy	Cost	Trade-off
Cheapest	₹18,050	Risk: High
Balanced	₹18,420	Risk: Low
Fastest	₹18,760	Delivery: 1 hour earlier

Actual displayed values should be generated by the prototype dataset rather than presented as pre-existing results.

23 Controlled Agentic Commerce

The merchant defines policies:



The agent can act inside predefined boundaries. This is controlled agentic commerce rather than unrestricted autonomous purchasing.

24 Cash-Aware Procurement

Supplier A	Supplier B
₹100,000	₹101,500
Immediate Payment	7-Day Payment Terms

If merchant liquidity is constrained, Supplier B may produce better economic utility.

$$\text{Procurement Utility} = \text{Product Cost} + \text{Logistics} + \text{Risk} + \text{Liquidity}$$

The financial layer should initially remain procurement-aware rather than becoming a lending product. Future financing can be enabled through regulated partners using verified procurement behavior as a signal.

25 One Payment Experience

Checkout		Backend settlement can distribute amounts to suppliers through appropriate payment infrastructure. For a hackathon prototype, payment can remain sandboxed or simulated.
Products	₹17,800	
Consolidated Delivery	₹420	
Other Charges	₹200	
TOTAL	₹18,420	
CONFIRM & PAY		

26 Predictive Stockout Engine

Calculation	System
Average Consumption	20 kg/day
Current Inventory	35 kg
Estimated stockout: 35 / 20	1.75 days
	⚠ Rice may run out in 1–2 days. Recommended order: 80 kg

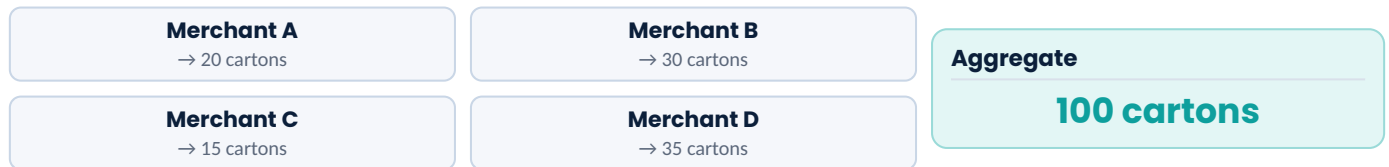
This changes the product from reactive procurement to predictive procurement.

27 Regional Demand Intelligence



At network scale, aggregate demand becomes a signal that can help suppliers prepare inventory before shortages become widespread.

28 Collective Procurement

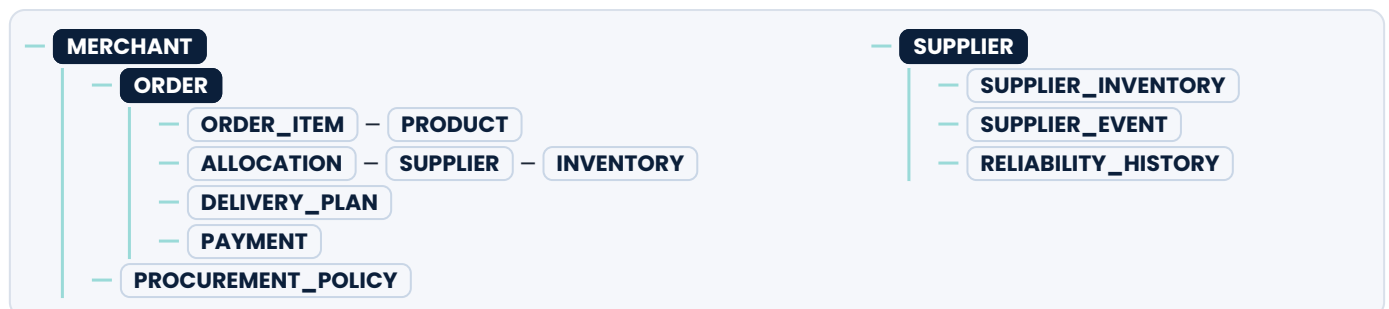


Potential benefits include wholesale pricing, fewer delivery trips and improved supplier utilization. This remains a scale/phase-2 capability.

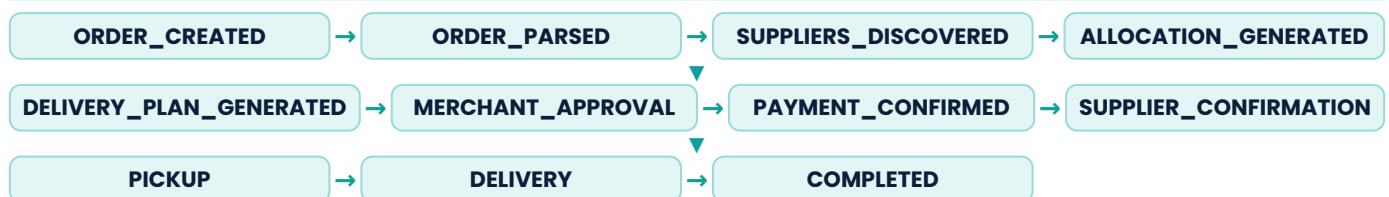
29 Database Architecture

Entity	Core Fields
Merchant	merchant_id, name, location, preferences, budget, procurement_policy
Product	product_id, brand, category, pack_size, unit, aliases
Supplier	supplier_id, location, operating_hours, payment_terms, reliability_score
SupplierInventory	supplier_id, product_id, available_quantity, price, timestamp, confidence
Order	order_id, merchant_id, status, deadline, total_cost, confidence
OrderItem	order_id, product_id, quantity
Allocation	order_id, supplier_id, product_id, quantity, price
DeliveryPlan	order_id, vehicle_id, route, stops, ETA, capacity
SupplierEvent	supplier_id, event_type, accepted, rejected, shortfall, ready, delayed
Payment	order_id, amount, status, settlement_reference

29.1 Relational Model



30 Event-Driven Order Lifecycle



Failure path



31 Technical Stack

Layer	Technology
Frontend	React / Next.js
Backend	FastAPI
Database	PostgreSQL
Cache / Event Processing	Redis
AI / Language	Sarvam APIs + LLM
Optimization	Google OR-Tools / CP-SAT
ML	LightGBM / XGBoost
Maps	Google Maps / Mapbox
Messaging	WhatsApp Business API / Simulator
Payments	UPI / payment sandbox or simulation
Deployment	Docker + Cloud

32 Backend Service Architecture

backend/

api/ - merchant.py - order.py - supplier.py - payment.py - delivery.py	ai/ - speech.py - intent.py - sku_normalizer.py - orchestrator.py	optimization/ - basket_optimizer.py - landed_cost.py - route_optimizer.py - replanner.py	supplier/ - discovery.py - inventory.py - reliability.py
risk/ - fulfillment_confidence.py - failure_detection.py	payments/ - checkout.py - settlement.py	events/ - publisher.py - handlers.py	database/ - models.py - repositories.py

33 API Design

Create Order

POST /api/orders

```
{
  "merchant_id": "M001",
  "input_type": "voice",
  "language": "kn",
  "transcript": "Anna naale 50 kilo rice beku",
  "deadline": "2026-10-06T10:00:00"
}
```

Response:

```
{
  "order_id": "ORD1042",
  "status": "PARSED",
  "items": [
    {
      "product_id": "RICE-001",
      "quantity": 50,
      "unit": "kg"
    }
  ]
}
```

Generate Procurement Plan

POST /api/orders/{order_id}/optimize

Response:

```
{
  "order_id": "ORD1042",
  "status": "OPTIMIZED",
  "total_cost": 18420,
  "fulfillment_confidence": 0.96,
  "suppliers": [...],
  "delivery_plan": {...}
}
```

Supplier Event

POST /api/supplier-events

```
{
  "supplier_id": "SUP-A",
  "order_id": "ORD1042",
  "event": "SHORTFALL",
  "expected_quantity": 100,
  "available_quantity": 40
}
```

Response:

```
{
  "status": "REPLANNING",
  "shortfall": 60
}
```

Replan

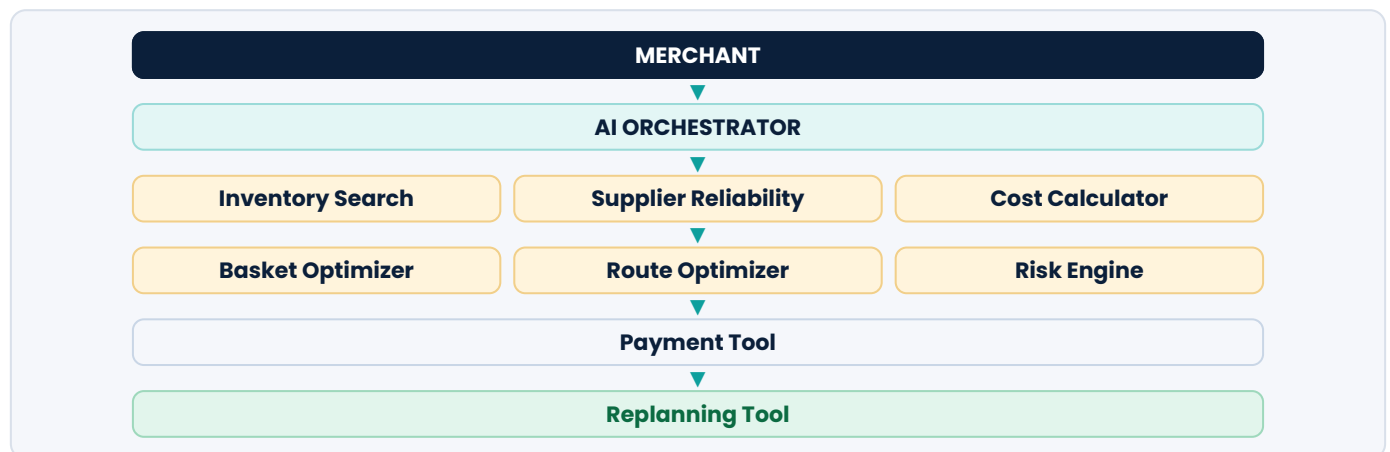
POST /api/orders/{order_id}/replan

Output:

```
{
  "status": "REPLANNED",
  "new_allocation": [...],
  "new_route": {...},
  "new_confidence": 0.95
}
```

34 AI Orchestrator

The platform should use one primary orchestration agent rather than many independent agents.



35 Tool Contracts

get_inventory()

```
{
  "product_id": "RICE-001",
  "location": "Bengaluru",
  "required_quantity": 100
}
```

Returns authoritative supplier inventory.

get_supplier_reliability()

```
{
  "supplier_id": "SUP-A"
}
```

Returns:

```
{
  "availability_accuracy": 0.96,
  "on_time_rate": 0.97,
  "cancellation_rate": 0.01,
  "shortage_rate": 0.02
}
```

calculate_landed_cost()

Combines product cost, delivery cost, risk, consolidation and liquidity.

optimize_basket()

Inputs: required items, supplier inventory, prices, reliability, MOQ, budget and policies. Output: optimal supplier allocation.

optimize_route()

Inputs: supplier locations, merchant location, vehicle capacity, ready times, delivery deadline and package requirements. Output: route, ETA, distance and delivery cost.

replan_order()

Inputs: existing plan, failure event, remaining requirements and current supplier state. Output: new allocation, new route and new confidence.

36 Security and Trust Model

Intent



Policy Check



Authorization



Tool Execution



Audit Log

The AI cannot exceed merchant budget, quantity limits, procurement policy, supplier authorization or payment authorization.

AI wants to purchase ₹24,000 | Merchant limit = ₹20,000 → **POLICY REJECTED**

37 Auditability

```
{
  "event_id": "EVT-001",
  "order_id": "ORD-1042",
  "timestamp": "...",
  "actor": "system",
  "action": "SUPPLIER_FAILOVER",
  "reason": "SHORTFALL",
  "previous_supplier": "SUP-A",
  "replacement_supplier": "SUP-D"
}
```

Every significant action should generate an event for debugging, merchant transparency, operational analysis, dispute investigation, reliability scoring and system evaluation.

38 Failure Handling

Failure	System Response
Supplier rejection	Find alternatives → reallocate → recalculate route
Partial inventory	Calculate shortfall → source remaining quantity
Price change	Recalculate landed cost → compare alternatives
Delivery infeasible	Re-optimize route
Merchant quantity change	Update basket → re-optimize
Deadline change	Recalculate feasibility
Alternative more expensive	Present trade-offs
No supplier can fulfill	Never hallucinate fulfillment
Substitution disabled	Do not substitute
Payment failure	Keep order in controlled state

39 No-Fulfillment Condition

⚠ **Full basket cannot currently be fulfilled.**

Rice Required	100 kg	Alternative:	Expected Delay:
Available	80 kg	70 kg substitute	2 hours

The merchant makes the final decision. The platform must never pretend that unavailable inventory exists.

40 Prototype Data Model



For the hackathon, physical suppliers, physical inventory, physical vehicles and real settlement infrastructure can be simulated. The optimization algorithm and AI interaction remain real.

41 Demo Dataset

Supplier	Rice	Oil	Biscuits	Distance	Reliability
A	60 kg	20 L	0	2.3 km	97%
B	40 kg	0	0	4.1 km	94%
C	0	30 L	20 cartons	3.2 km	96%
D	20 kg	20 L	10 cartons	5.0 km	91%

Merchant requirement

100 kg Rice, 50 L Oil, 20 Biscuit Cartons.

Initial plan

A → 60 kg Rice + 20 L Oil
 B → 40 kg Rice
 C → 30 L Oil + 20 Cartons

42 Primary Demonstration

Step 1 Natural Language Input

Merchant speaks a local-language voice order.

Step 2 AI Parsing

ORDER #1042
 Rice 100 kg · Cooking Oil 50 L · Biscuits 20 cartons
 Deadline Tomorrow 10 AM

Step 3 Supplier Discovery

Rice: A → 60 kg, B → 40 kg
 Oil: A → 20 L, C → 30 L
 Biscuits: C → 20 cartons

Step 4 Optimization

A → 60 kg Rice + 20 L Oil
 B → 40 kg Rice
 C → 30 L Oil + 20 Cartons

Step 5 Delivery Optimization

The system calculates the best delivery arrangement.

Step 6 Trigger Failure

SUPPLIER A STOCK SHORTAGE
 Expected: 60 kg · Actual: 40 kg

Step 7 Automatic Failerover

Shortfall = 20 kg
 D → 20 kg

Step 8 Reoptimization

Old: A → 60 kg · B → 40 kg
New: A → 40 kg · B → 40 kg · D → 20 kg

Step 9 Route Recalculation

The route optimizer recalculates the route based on the new supplier allocation.

Step 10 Merchant Notification

Supplier A confirmed only 40 kg. 20 kg shortfall detected. Backup Supplier D activated. Fulfillment plan re-optimized. Updated delivery plan ready.

43 Demo WOW Moments

WOW	Demonstration
1 — India-first interface	Merchant speaks a local-language voice order and system understands it.
2 — Optimization	Show the computed allocation and fulfillment plan.
3 — Failure	Simulate supplier failure and show backup supplier activation.
4 — Confidence recovery	Show confidence change after failure and recovery.

44 Prototype Screens

- Merchant voice/WhatsApp interface
 - Parsed basket
 - Supplier comparison
 - Optimization result
 - Delivery route
- Supplier failure event
 - Automatic reallocation
 - Updated route
 - Merchant confirmation/payment
 - Control tower

45 Merchant Control Tower

SHOP SUPPLY HEALTH

Basket Fulfillment

97.8%

Stockout Risk

LOW

Supplier Reliability

94%

Delivery Trips / Order

1.2

Procurement Time

3m 18s

Cash Pressure

MEDIUM

⚠ Oil supply tightening

✓ Rice backup supplier active

The dashboard positions the product as a merchant supply operating system rather than an AI chatbot.

46 Supplier Dashboard

NEW ORDER

Rice

60 kg

Oil

20 L

Required

Tomorrow 8–10 AM

ACCEPT

MODIFY

REJECT

If a supplier modifies 60 kg to 40 kg, the system immediately calculates the shortfall and triggers re-optimization.

47 Performance Evaluation

Baseline	Description	Evaluation Metrics	
Baseline 1	Nearest supplier	• Total Procurement Cost	• Supplier Failure Recovery Rate
Baseline 2	Cheapest supplier	• Delivery Distance	• Procurement Time
Baseline 3	Single supplier	• Number of Deliveries	• Average Replanning Latency
Baseline 4	Manual/random allocation	• Fulfillment Rate	• Optimization Runtime
Proposed	Joint procurement + fulfillment optimization	• Deadline Success Rate	• Order Parsing Accuracy
		• Stockout Recovery Time	• SKU Normalization Accuracy

48 Pilot Hypotheses

The following should be presented as targets/hypotheses, not achieved results unless measured:

Metric	Pilot Hypothesis
Procurement time	50–70% reduction
Basket fulfillment	>95%
Stockout-related lost sales	20–30% reduction
Fragmented delivery trips	15–30% reduction
Effective landed cost	3–8% reduction
Manual supplier coordination	50%+ reduction

49 Competitive Positioning

The product should not claim that no one is solving merchant procurement. Adjacent players include Udaan, Jumbotail, ONDC / DigiDukaan, WhatsApp/AI ordering systems and PANTR.

Differentiation: Existing systems digitize procurement and distribution. Stockout Firewall focuses specifically on dynamic procurement resilience—continuously re-solving supplier allocation and fulfillment when real-world conditions change.

50 Differentiation Matrix

Capability	Traditional Procurement	Stockout Firewall
Supplier Discovery	✓	✓
Ordering	✓	✓
Multilingual Voice	Limited / varies	✓
SKU Normalization	Limited	✓
Reliability-Aware Procurement	Limited	✓
Basket Optimization	Limited	✓
Total Landed Cost	Limited	✓
Supplier Failover	Limited	✓
Dynamic Replanning	Limited	✓
Route Optimization	Varies	✓
Fulfillment Confidence	Limited	✓
Explainable Procurement	Limited	✓
Cash-Aware Procurement	Limited	✓
Predictive Stockout	Future / varies	✓
Controlled Agent Commerce	Limited	✓

51 Business Model

- **Transaction fee:** small percentage on successfully fulfilled orders.
- **Merchant subscription:** premium predictive stockout, analytics, autonomous policies and cash-flow intelligence.
- **Logistics orchestration margin** where appropriate.
- **Future financial services** through regulated partners.

52 Go-to-Market

20 Merchants	10–20 Suppliers	1 Local Delivery Network	3–5 km Radius
-----------------	--------------------	-----------------------------	------------------

Baseline period: measure current procurement time, cost, delivery trips, stockouts and supplier failures.

Pilot period: deploy Stockout Firewall and compare before versus after.

53 Scalability Roadmap



At scale, millions of merchant interactions can create regional demand signals that improve supply intelligence.

54 Long-Term Supply Chain Weather Forecast



Potential signals include demand velocity, inventory, supplier reliability, historical patterns, seasonal events and local demand.

55 Long-Term Product Vision

The platform ultimately becomes the operating system for merchant procurement.

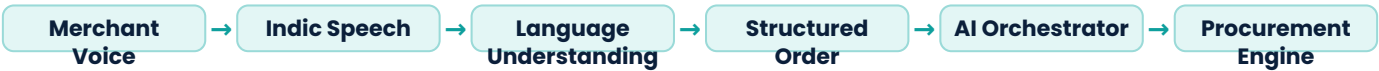
Agent boundaries

Budget	₹20,000/day
Reliability	>90%
Price Tolerance	3%
Preferred Brands	ON
Auto-Failover	ON
Auto-Substitution	OFF
Delivery Deadline	10 AM

The agent continuously manages procurement within these boundaries. This is the final expression of controlled agentic commerce.

56 Why Sarvam Is Technically Necessary

Sarvam should not appear merely as a sponsor or logo integration. Its role is to solve the actual interaction problem: the merchant should not need to learn another digital procurement interface.



Sarvam serves as the India-first voice/language layer, while procurement intelligence remains in the orchestration and optimization system.

57 Technical Capabilities Demonstrated

- Multilingual Speech
 - LLM Tool Calling
 - Information Extraction
 - Entity Resolution
- SKU Normalization
 - Supplier Graph Modeling
 - Recommendation Systems
 - Constraint Optimization
- Vehicle Routing
 - Event-Driven Architecture
 - Real-Time Replanning
 - Predictive ML
- Payment Orchestration
 - Explainable AI
 - Backend Engineering
 - Policy-Based Agent Control

58 What Should Actually Be Built for MVP

Must Build

- ✓ Voice/Text Input
- ✓ Multilingual Parsing
- ✓ SKU Normalization
- ✓ Supplier Database
- ✓ Inventory Database
- ✓ Reliability Score
- ✓ Basket Optimizer
- ✓ Landed Cost Calculator
- ✓ Route Optimizer
- ✓ Supplier Failure Simulator
- ✓ Automatic Reallocation
- ✓ Route Reoptimization
- ✓ Merchant Approval
- ✓ Payment Simulation
- ✓ Control Tower

Can Remain Simulated

- Physical suppliers
- Real inventory
- Physical vehicles
- Actual logistics
- Payment settlement
- Large-scale merchant network

Phase 2

- Predictive Stockout
- Collective Procurement
- Regional Demand Intelligence
- Supply Chain Weather Forecast

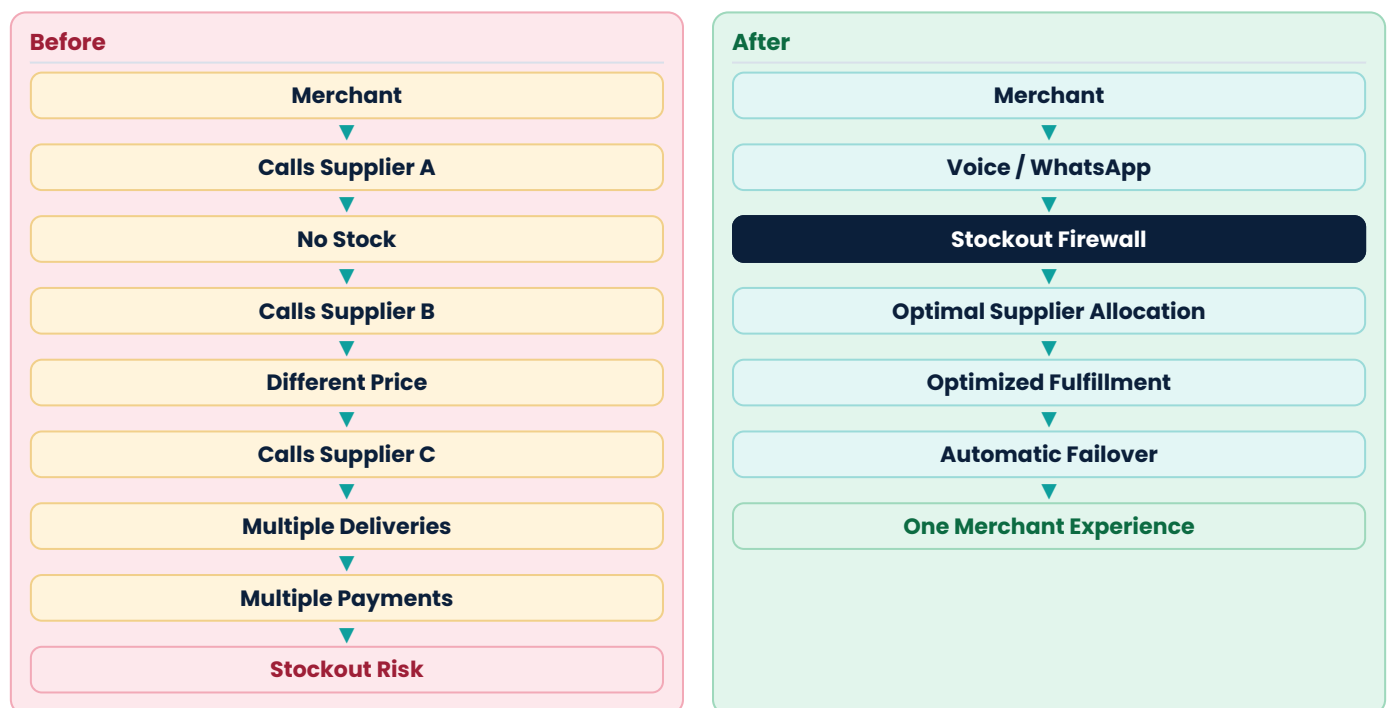
59 Three-Minute Demonstration Flow

Time	Action
0:00–0:20	Merchant voice/WhatsApp order
0:20–0:40	Convert voice to structured basket
0:40–1:00	Supplier intelligence finds candidates
1:00–1:20	Optimization engine produces allocation
1:20–1:40	Fulfillment engine generates delivery plan
1:40–2:00	Supplier fails
2:00–2:20	Backup supplier automatically activated
2:20–2:40	Route re-optimized
2:40–3:00	Merchant receives updated plan/payment view

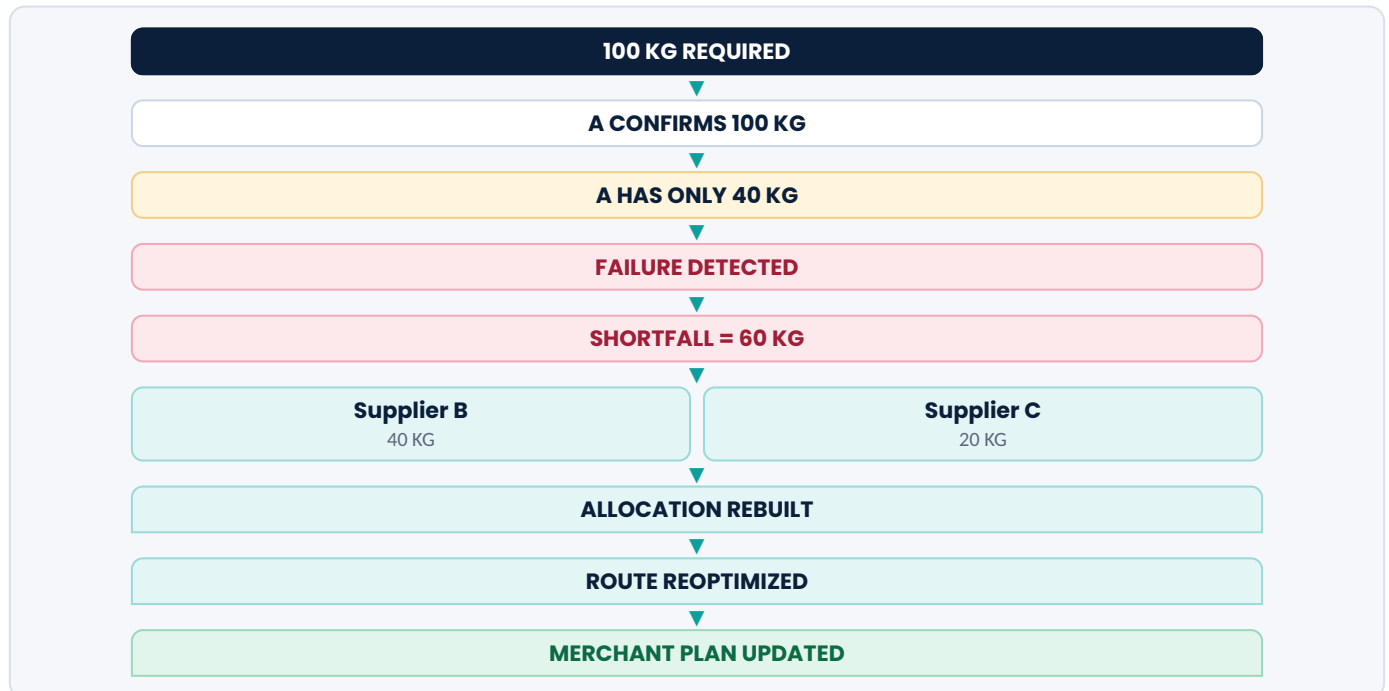
60 Recommended Presentation Flow

Slide	Content
1	STOCKOUT FIREWALL — When one supplier fails, the merchant shouldn't.
2	Problem
3	Merchant's current fragmented workflow
4	Why basic digital procurement is insufficient
5	Stockout Firewall solution
6	AI procurement engine
7	Supplier failover architecture
8	Dynamic fulfillment optimization
9	System architecture
10	Live prototype
11	Technical differentiation
12	Measured impact
13	Business model + feasibility
14	Scalability + future vision

61 Critical Before / After



62 Core Failure Demonstration



This is the central technical differentiator of Stockout Firewall.

63 Final Technical Differentiator

"We don't simply help merchants find suppliers; we continuously compute the most reliable way to fulfill their basket despite supplier, inventory and delivery uncertainty."

64 Final Product Definition

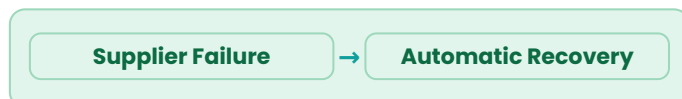
Stockout Firewall is an AI procurement resilience platform for India's small merchants.



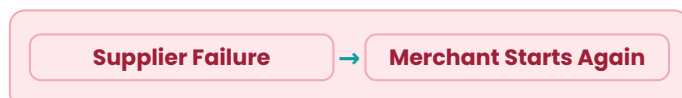
Merchant experience



And when reality changes:



rather than:



That is the technical essence of Stockout Firewall.

A Appendix A — Implementation Principles

- Keep authoritative facts outside the LLM.
- Use deterministic optimization for allocation and routing.
- Use event-driven state transitions for dynamic changes.
- Make supplier failure a first-class system event.
- Treat the procurement plan as mutable until fulfillment completion.

- Enforce merchant policies before execution.
 - Record significant actions in an audit trail.
 - Use simulated infrastructure where real-world infrastructure is unnecessary for the prototype.
- Present targets as hypotheses until measured.
 - Keep dynamic procurement resilience as the central differentiator.

B Appendix B — One-Line Architecture

